

SMM: Standard Model of Models — Foundation

v1.1 (Math-Clean)

This specification defines a substrate-agnostic, schema-native kernel for models, interfaces, morphisms, contracts, and proof obligations. All MUSTs are normative unless marked informative.

1. Conformance Levels

Conformance is declared at the package level:

- **core**: packages, imports, models, interfaces, morphisms, canonicalization, structural identity.
- **contracts**: contracts, assertions, proof obligations, verifier routing.
- **smm-imd-1**: normative binding to IMD verifiers and audit.

`package.conformance` MUST include at least `core`. Profiles MAY add further constraints.

2. Canonicalization and Structural Identity (Normative)

Canonical form: JSON Canonicalization Scheme (JCS, RFC 8785). All SMM artifacts are normalized with JCS prior to hashing or comparison.

Digest: `structuralDigest = base64url(SHA-256(JCS(bytes)))`.

Identity rule: two artifacts are identical iff their `structuralDigest` values match. Toolchains MUST compute digests; storage MAY elide them.

Normalization scope: entire artifact objects, excluding transport metadata (e.g., repository tags).

3. Imports and Fixpoint Semantics

- Import graph is acyclic by default.
- Cycles are permitted only when the package declares non-`none` fixpoint semantics.
- Resolution order is topological; with cycles, evaluate by declared fixpoint.

3.1 Package-level Fixpoint Declaration

```
{
  "fixpointSemantics": {
    "type": "string",
```

```

    "enum": ["none", "least", "greatest"],
    "default": "none"
  }
}

```

- `least`: interpret recursive specifications via least fixpoint.
- `greatest`: via greatest fixpoint.

A proof obligation of kind `refinement/fixpoint` MUST exist for any package with `fixpointSemantics != "none"`.

4. Common \$defs

These definitions are reused across schemas.

```

{
  "$defs": {
    "NonEmptyString": { "type": "string", "minLength": 1 },
    "SemVer": {
      "type": "string",
      "pattern": "^(0|[1-9]\\d*)\\.(0|[1-9]\\d*)\\.(0|[1-9]\\d*)(?:-([0-9A-Za-z-\\.]+)?(?:\\+[0-9A-Za-z-\\.]+)?)?$"
    },
    "UUID": { "type": "string", "format": "uuid" },
    "DateTime": { "type": "string", "format": "date-time" },
    "Any": {},
    "Option": { "oneOf": [{ "type": "null" }, { "$ref": "#/$defs/Any" }] },
    "Result": {
      "oneOf": [
        { "type": "object", "required": ["ok"], "properties": { "ok": { "$ref": "#/$defs/Any" } }, "additionalProperties": false },
        { "type": "object", "required": ["err"], "properties": { "err": { "$ref": "#/$defs/Any" } }, "additionalProperties": false }
      ]
    }
  }
}

```

5. Path Syntax (Normative)

All intra-artifact paths use **JSON Pointer** (RFC 6901). Relative pointers are resolved against the artifact root unless otherwise stated. Toolchains MUST validate pointers on ingestion.

6. Core Schemas

All core schemas set `additionalProperties: false`. Fields not specified here are disallowed.

6.1 package

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "smm.package",
  "type": "object",
  "required": ["name", "version", "conformance", "models"],
  "properties": {
    "name": {"$ref": "#/$defs/NonEmptyString"},
    "version": {"$ref": "#/$defs/SemVer"},
    "conformance": {
      "type": "array",
      "minItems": 1,
      "items": {"type": "string", "enum": ["core", "contracts", "smm-imd-1"]},
      "uniqueItems": true
    },
    "fixpointSemantics": {"type": "string", "enum": ["none", "least",
"greatest"], "default": "none"},
    "imports": {"type": "array", "items": {"$ref": "#/definitions/import-ref"}},
    "models": {"type": "array", "minItems": 1, "items": {"$ref": "#/definitions/
model"}},
    "interfaces": {"type": "array", "items": {"$ref": "#/definitions/
interface"}},
    "morphisms": {"type": "array", "items": {"$ref": "#/definitions/morphism"}},
    "contracts": {"type": "array", "items": {"$ref": "#/definitions/contract"}},
    "proofObligations": {"type": "array", "items": {"$ref": "#/definitions/
proof-obligation"}}
  },
  "additionalProperties": false,
  "definitions": {}
}
```

6.2 import-ref

```
{
  "title": "smm.import-ref",
  "type": "object",
  "required": ["package", "versionRange"],
  "properties": {
    "package": {"$ref": "#/$defs/NonEmptyString"},

```

```

    "versionRange": {"type": "string"},
    "integrity": {"type": "string"}
  },
  "additionalProperties": false
}

```

6.3 model

```

{
  "title": "smm.model",
  "type": "object",
  "required": ["name", "schema"],
  "properties": {
    "name": {"$ref": "#/$defs/NonEmptyString"},
    "schema": {"$ref": "#/$defs/Any"},
    "doc": {"type": "string"}
  },
  "additionalProperties": false
}

```

6.4 interface

```

{
  "title": "smm.interface",
  "type": "object",
  "required": ["name", "methods"],
  "properties": {
    "name": {"$ref": "#/$defs/NonEmptyString"},
    "methods": {
      "type": "array",
      "minItems": 1,
      "items": {"$ref": "#/definitions/method"}
    }
  },
  "additionalProperties": false,
  "definitions": {
    "method": {
      "title": "smm.method",
      "type": "object",
      "required": ["name", "params", "returns"],
      "properties": {
        "name": {"$ref": "#/$defs/NonEmptyString"},
        "params": {"$ref": "#/$defs/Any"},
        "returns": {"$ref": "#/$defs/Any"},
        "effects": {"type": "array", "items": {"type": "string"}}
      }
    }
  }
}

```

```

    },
    "additionalProperties": false
  }
}
}

```

6.5 morphism

```

{
  "title": "smm.morphism",
  "type": "object",
  "required": ["name", "from", "to", "map"],
  "properties": {
    "name": {"$ref": "#/$defs/NonEmptyString"},
    "from": {"$ref": "#/$defs/NonEmptyString"},
    "to": {"$ref": "#/$defs/NonEmptyString"},
    "map": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["source", "target"],
        "properties": {
          "source": {"type": "string", "description": "JSON Pointer into source model"},
          "target": {"type": "string", "description": "JSON Pointer into target model"}
        },
        "additionalProperties": false
      }
    },
    "laws": {"type": "array", "items": {"type": "string"}}
  },
  "additionalProperties": false
}

```

6.6 contract

```

{
  "title": "smm.contract",
  "type": "object",
  "required": ["name", "subjectRef", "assertions"],
  "properties": {
    "name": {"$ref": "#/$defs/NonEmptyString"},
    "subjectRef": {"type": "string", "description": "JSON Pointer to model/"

```

```

interface/morphism"},
  "assertions": {"type": "array", "items": {"$ref": "#/definitions/
assertion"}}
},
"additionalProperties": false,
"definitions": {
  "assertion": {
    "title": "smm.assertion",
    "type": "object",
    "required": ["lang", "expr"],
    "properties": {
      "lang": {"type": "string", "enum": ["jsonlogic", "smtlib2"]},
      "expr": {"$ref": "#/$defs/Any"},
      "vars": {
        "type": "object",
        "additionalProperties": {"type": "string", "description": "JSON
Pointer binding"}
      }
    },
    "additionalProperties": false
  }
}
}
}

```

6.7 proof-obligation

```

{
  "title": "smm.proof-obligation",
  "type": "object",
  "required": ["name", "kind", "verifierRef"],
  "properties": {
    "name": {"$ref": "#/$defs/NonEmptyString"},
    "kind": {"type": "string"},
    "subjectRef": {"type": "string", "description": "JSON Pointer to subject"},
    "verifierRef": {"type": "string", "format": "uri"},
    "witnessRef": {"type": "string", "format": "uri"}
  },
  "additionalProperties": false
}

```

7. Semantics

- **Well-typed morphisms:** every `map[*]` pointer MUST resolve; mapped fields MUST be type-compatible under the subject models' JSON Schemas. Implementations SHALL perform static checks when schemas are available.
 - **Import closure:** packages MUST resolve all `imports` before evaluation.
 - **Evaluation context:** assertion variables bind to pointers within the `subjectRef` resolution context.
 - **Determinism:** canonicalization precedes all hashing and equality checks.
-

8. Acceptance Tests (Normative)

1. **Canonical round-trip:** parse → JCS → hash → compare across toolchains; hashes MUST match.
 2. **Import validation:** enforce version ranges and acyclicity when `fixpointSemantics = "none"`.
 3. **Morphism checker:** pointer resolvability, type preservation, and `laws` obligations satisfied.
 4. **Assertion runner:** execute `jsonlogic` or `smtlib2` with declared `vars` bindings.
 5. **Fixpoint packages:** presence of `refinement/fixpoint` obligation and successful verification.
-

9. SMM-IMD Profile (`smm-imd-1`)

Scope: binds verifiers and audit to IMD components.

- `proof-obligation.verifierRef` MUST be one of:
 - `imd:ace` for Analytical Certificate Engine checks
 - `imd:ace:<name>` for a named verifier registered in ACE
 - `witnessRef` MUST be an `imd:archivum:<record-id>` URI when a witness is produced.
 - Contracts SHOULD use `jsonlogic` for local checks and `smtlib2` for solver-backed proofs.
 - Tooling MUST emit an Archivum record for each successful obligation under this profile.
-

10. Security and Portability

- Only `jsonlogic` and `smtlib2` assertion languages are allowed in this spec. Engine-native or WASM evaluators are out of scope.
 - JSON Pointer eliminates evaluator ambiguities common in query languages.
 - Canonicalization and hashing mitigate supply-chain drift across repositories.
-

11. Minimal Worked Example (Informative)

```
{
  "name": "hello",
  "version": "1.0.0",
  "conformance": ["core", "contracts", "smm-imd-1"],
  "fixpointSemantics": "none",
  "models": [
    {"name": "User", "schema": {"type": "object", "required": ["id", "email"],
  "properties": {"id": {"type": "string", "format": "uuid"}, "email": {"type":
"string", "format": "email"}}}, "additionalProperties": false}}
  ],
  "interfaces": [
    {"name": "UserAPI", "methods": [
      {"name": "get", "params": {"type": "object", "properties": {"id":
{"type": "string", "format": "uuid"}}}, "required": ["id"],
"additionalProperties": false}, "returns": {"$ref": "#/models/0/schema"}}
    ]
  ],
  "morphisms": [
    {"name": "User_to_PublicUser", "from": "User", "to": "PublicUser", "map":
[{"source": "/email", "target": "/contact"}], "laws": ["morphism/law"]}
  ],
  "contracts": [
    {"name": "EmailIsValid", "subjectRef": "/models/0", "assertions": [
      {"lang": "jsonlogic", "expr": {"and": [{">": [{"var": "email_len"}, 3],
{"in": [{"@", {"var": "email"}}]}]}, "vars": {"email": "/schema/properties/email/
example", "email_len": "/schema/properties/email/minLength"}}
    ]
  ],
  "proofObligations": [
    {"name": "UserEmailFormat", "kind": "safety/invariant", "subjectRef": "/
models/0/schema/properties/email", "verifierRef": "imd:ace"}
  ]
}
```

End of v1.1